

An Integrated Algorithm for Geometric Cut, Tear and Drill of Rigged 3D Models

Bachelor Thesis



Author: John Petropoulos (Student ID: 4028)

Supervisor: Prof. Papagiannakis George

Co-Supervisor: Dr. Kamarianakis Manos

Department of Computer Science

School of Sciences & Engineering

University of Crete

Heraklion, Greece

September 19, 2024

Abstract

In this work, we present an integrated algorithm for geometric cut, tear and drill of rigged 3D models inside the popular game engine Unity3D. Our algorithms, allow the user to perform cuts, tears and drills in any place on the surface of a 3D model in real time, without requiring any heavy pre - processing. These operations can be done either in a Virtual Reality or in a regular 3D Desktop environment. Furthermore, our procedures allow further re-animation and re-deformation of the skinned 3D model.

Declaration of Authorship

I, John Petropoulos, declare that this thesis titled, "An Integrated Algorithm for Geometric Cut, Tear and Drill of Rigged 3D Models", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: John Petropoulos

Date: September 19, 2024

Acknowledgements

I would like to express my gratitude to my supervising professor George Pappagiannakis for his continuous support and essential direction throughout the course of this project.

I would also like to thank Dr. Manos Kamarianakis, Dr. Antonis Protopsaltis, Mr. Nick Lydatakis and Mr. Steve Kateros for their instructions and helpful advice.

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Scope and Objectives	1
1.3	State of the Art	1
1.4	Achievements	2
1.5	Thesis Overview	2
2	Cutting Algorithm	3
2.1	First Implementation	3
2.2	Further Optimizations and Improvements	5
2.3	Running Times for Cutting	5
2.3.1	First Implementation Running Times	5
2.3.2	After Optimization Running Times	7
3	Tearing Algorithm	7
3.1	Running Times for Tearing	9
3.1.1	Offline preprocessing running times	9
3.1.2	Real-Time Tearing Running Times	10
4	Drilling Algorithm	10
4.1	Dense	10
4.2	Drill	11
4.3	Running Times for Drilling	13
5	Conclusions, Limitations and Future Work	15
	References	17
A	Manual and Instructions	19
A.1	Unity Installation	19
A.2	Unity Project Creation	21
A.3	Import Models, Prefabs and Scripts Into Unity Project	23

List of Figures

1	(Left)Original model, (Middle) Single CTD on model, (Right) Multiple CTDs on model.	2
2	A cut, drilled and teared cactus that has been re-deformed.	3
3	Different size drills and tears with some cuts on the cactus model.	4
4	(Top) Cutting plane, (Middle) Vertices on negative, positive side and on the plane, (Bottom) Cut arm model.	6
5	(Left) In between intersection points, (Right) Tear segment, (Bottom) Opened Tear segment.	9
6	(Left) Middle point of each edge, (Middle) New triangle consisting of the new edges, (Right) New triangles.	11
7	(Left) Vertex seperation, (Right) Affected ids of the drilling.	12
8	(Left) Drill intersection points, (Right) Complete drilled hole.	12
9	Press the button in the red rectangle and follow the instructions.	19
10	Press the button in the red rectangle to see the current installs.	19
11	Press the button in the red rectangle to add a new Unity version.	20
12	Press the button in the red rectangle to choose the correct version.	20
13	Press the button in the red rectangle to begin the installation.	21
14	Press the button in the red rectangle to show available versions.	21
15	Press the button in the red rectangle to select the newly installed version.	22
16	Create the project with the correct setting.	22
17	Drag and drop the assets inside unity.	23
18	Drag and drop the prefabs into the hierarchy panel.	23

List of Tables

1	Specific running times for each case of cutting.	7
2	Specific running times for each case of cutting with optimizations.	8
3	Specific running times for each case for tearing.	10
4	Specific running times for each case for tearing in real time. . . .	10
5	The above running times are without the offline calculations that happen before the first frame(i.e. triangulation, neighbors, etc). . .	14
6	Specific running times for each case.	15
7	Specific running times on parallelized functions.	15
8	Drilling running times.	15

1 Introduction

1.1 Background and Context

Animations on a rigged model have been an important area of research in Computer Graphics since inception (Magenat-Thalmann, Laperrire, & Thalmann, 1988). This interest has been further amplified by the development and rise of Virtual Reality (VR) as well as the increasing popularity of Video Games. With VR becoming mainstream, the hardware required to experience it also become cheaper, both for personal/commercial use but also for industrial purposes. This bloom inevitably created the need for more complex operations in VR environments that would facilitate immersion and provide a satisfying user experience via tools that would bridge the gap between what a character could do in reality and what a virtual character could do in Virtual Reality.

Examples of such operations involve the ability to perform cuts, tears, and drills on the surface of a 3D model. Such operations are extremely helpful especially in the context of various VR applications such as the ones related to VR Surgery education. Of course, to further enhance user immersion, such operations would have to be performed in real-time and if possible allow further deformation for skinned models.

1.2 Scope and Objectives

The main goal of this work is to implement three algorithms that would allow the user to cut, tear and drill the surface of a 3D rigged models within a modern VR engine, in our case Unity3D. A key objective was the implementation in Unity C# scripts of the algorithms presented in (Kamarianakis & Papagiannakis, n.d.). The redesign of the original algorithms, initially written in Python, was done bearing in mind that we wanted the final modules to produce real-time, or near real-time, results when run in Virtual Reality or regular 3D desktop environment.

1.3 State of the Art

The algorithms that exist thus far (Bruyns et al., 2002; Wu, Westermann, & Dick, 2015), use tetrahedral mesh representation of the model, which demands a high-cost preprocessing to be performed. Since their inception, such algorithms and procedures have been optimized and polished in order to achieve real-time results (Bielser, Glardon, Teschner, & Gross, 2004; Mor & Kanade, 2000; Bruyns & Senger, 2001). Soft body meshes have also been used by many engines to increase realism of cuts, at the expense of performance (Bielser, Maiwald, & Gross,

1999; Bender, Müller, Otaduy, Teschner, & Macklin, 2014; Berndt, Torchelsen, & Maciel, 2017).

1.4 Achievements

In this thesis, we present three optimized algorithms for Cutting, Tearing and Drilling (CTD) on rigged and deformable models that can be further re-animated and re-deformed, all done in real-time inside the Unity 3D engine. To achieve minimal running times, we used parallelism whenever possible and also created useful data-structures that were populated during the model initial import. In this way, we managed to reduce most of the online running times using a suitable preprocessing, as the updates of these data structures after each CTD operation require only a fraction of the initialization time.

We illustrate convincing results in Figures 1-3 and the video accompanying this thesis, available at <https://bit.ly/3l3XV5Q>. These results are part of a publication in progress.



Figure 1: (Left) Original model, (Middle) Single CTD on model, (Right) Multiple CTDs on model.

1.5 Thesis Overview

An overview of this thesis is the following: In Sections 2, 3 and 4 we present the algorithms/procedures and running times for Cutting, Tearing and Drilling on the surface of rigged 3D models respectively. In section 5, we state our conclusions and final thoughts, also we outline some further optimization and extensions that can be done. In the appendix section, we describe the steps required to setup and run the Cut/Tear/Drill (CTD) project in Unity3D.



Figure 2: A cut, drilled and teared cactus that has been re-deformed.

2 Cutting Algorithm

A rigged 3D model inside Unity consists of many data structures to enable its correct representation. The three fundamental ones are the following:

1. **Vertices Array:** An array of three dimensional vectors that represent the vertices/points of the model.
2. **Triangles Array:** An array of integers that contains indices into the vertex array. The size of the triangle array must always be a multiple of 3 and every continuous group of three cells of the triangles array represent a triangle on the model.
3. **BoneWeights Array:** An array of the BoneWeights data structure that must have the same size as the vertices array and is used to store the influence that each bone has to each vertex of the model.

2.1 First Implementation

These are the basic steps to implement the cutting algorithm:

1. First we have a plane (see Figure 4 - Top) that will determine where will the cut take place. Then we find the indices of the vertices that are located on the positive side of the plane, on the negative and the ones that are on the plane and we save them in three separate lists. (see Figure 4 - Middle)



Figure 3: Different size drills and tears with some cuts on the cactus model.

2. After that, each triangle in the mesh is categorized in 4 main categories, depending on the number of vertices that fall on the positive and negative sides of the cutting plane: 3P, 2P1N, 1P2N, 3N etc. Then, we calculate the intersection points of each of its edges with the plane and we retriangulate them accordingly. Also, here we store the new vertices (intersection points) to the mesh with their neighbor vertices in order to later calculate their new weights.
3. Afterwards, we update the weights of the new vertices that we have added to the mesh. Then we separate the new triangles/faces that are on the positive side the plane and on the negative side of the plane. Lastly, we update the triangles array of the mesh to contain only one of the two (see Figure 4 - Bottom).

2.2 Further Optimizations and Improvements

In order to find how many vertices of the triangle are on positive side, how many are on the negative and how many are on the plane, we have to perform a linear search of each triangle/face in the positive, negative and on-plane lists, which, depending on the 3D model's volume, can be quite large.

To optimize this procedure, instead of using a simple linked list to store the positive, negative and on-plane indices we used an array that takes the value 1 if the current index is on the positive side, the value -1 if the current index is on the negative side and 0 if it is on-plane.

As an index on that array we use the index of each vertex(vertex 0 will be on the 0 position on the table, vertex 1 will be on the 1 position of the table and so on). By doing that we don't have to search a huge list for every index of a vertex which takes $O(n)$ time, we just directly access an array using the index of the vertex and depending of the value of that index (1,-1,0) we determine on which side of the plane the current index of a vertex is, which takes $O(1)$ time.

Also, another optimization that we have done is to store the triangles that contain the newly calculated intersection points in order to reduce the complexity of the procedure SplitFaces that separates the indices later.

2.3 Running Times for Cutting

2.3.1 First Implementation Running Times

Specs: CPU: Intel core i9 9900k @ 3.60GHz, GPU: Nvidia RTX 2080, RAM: 16GB.

We are testing the following cases:

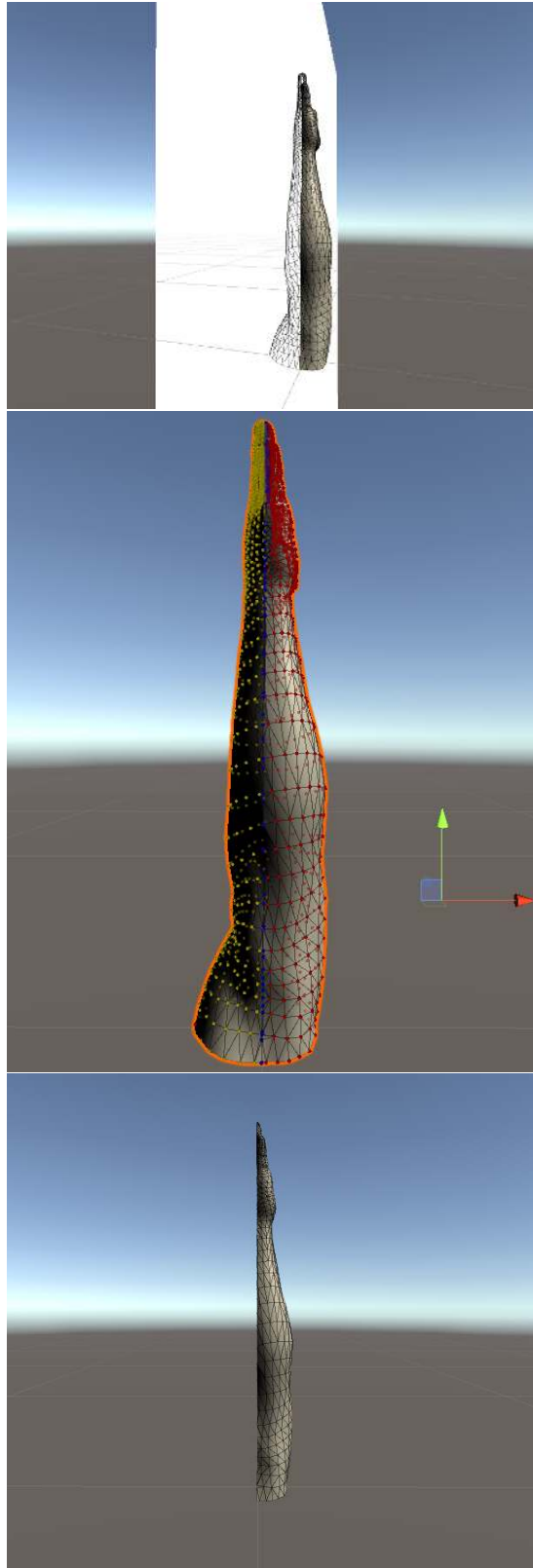


Figure 4: (Top) Cutting plane, (Middle) Vertices on negative, positive side and on the plane, (Bottom) Cut arm model.

- **Case 1:** Non-parallel and not dense.

In this case we used single-threaded execution of the code on the original arm model. Vertices: 2535, Triangles: 5037

- **Case 2:** Parallel and not dense..

In this case we we used multi-threaded execution of the code and we haven't applied any further triangulation to make the model more dense(i.e. we have the original arm model). Vertices: 2535, Triangles: 5037.

- **Case 3:** Non-parallel and dense.

In this case we used single-threaded execution of the code and we have applied triangulation to make the model more dense to every edge of every triangle. Vertices: 10109, Triangles: 20148.

- **Case 4:** Parallel and dense.

In this case we we used multi-threaded execution of the code and we have applied triangulation to make the model more dense to every edge of every triangle. Vertices: 10109, Triangles: 20148.

	Calculate Lists	Retriangulate	Update Weights
Case1	0.6793156s	1.26354s	0.003389s
Case2	0.02083s	1.25312s	0.0033426s
Case3	29.64207s	18.61102s	0.002616s
Case4	0.04492s	19.64084s	0.007444s
	Split faces	Remake triangles	Total
Case1	6.1945s	0.6688s	8.819s
Case2	0.57687s	0.00454s	1.863s
Case3	52.76247s	8.6107s	109.63s
Case4	5.147425s	0.01017s	24.8597s

Table 1: Specific running times for each case of cutting.

2.3.2 After Optimization Running Times

The running times that have changed with the optimization are the SplitFaces, Retriangulate and Overall, so we wont repeat the other since they are the same as above. Also the testing cases are the same as above. (see Table 2)

3 Tearing Algorithm

These are the basic steps to implement the tearing algorithm:

	Retriangulate	Split faces	Total
Case1	0.1403s	0.000555s	1.4938s
Case2	0.1633s	0.00265s	0.2210s
Case3	1.2263s	0.00145s	35.757s
Case4	1.2708s	0.00821s	1.3270s

Table 2: Specific running times for each case of cutting with optimizations.

1. First, we are given two segments, which indicate the starting and ending position of the scalpel that hypothetically tears the surface of the model.
2. We find the intersection points of those segments with the model and we also store the index of the triangle that they belong to.
3. Next, we create a plane that passes and contains the scalpel's final segment along with the initial intersection point. We then determine the intersection points of that plane with every edge of the model. In this step, we also store the triangles that contain or touch the intersection points.
4. After this is done, we have to separate and pick the intersection points that exist in between the two tear segments. (See Figure 5 - (Left)). To achieve this:
 - (a) We go to one of the initial tear points that we have found in step 1 and we find the two adjacent intersection points of the tear plane with edges of its triangle. We pick the one closer to the other tear point.
 - (b) After that, with help of an edge neighbors list, starting from that point that we have found above, we pick the intersection point that exists in the same triangle as the previous one but on another edge
 - (c) We continue this procedure until we come across the second tear point.
5. Now that we have found all the intersection points in between the tear axis we can begin the re-triangulate process:
 - (a) We iterate through the triangles of the in between intersection points and for each one we determine how many of its vertices are on the negative and how many are on the positive side of the tear plane.
 - (b) Depending on that, we do the proper re-triangulation, duplicating the intersection points so that the triangles on the negative side have different vertices from those on the positive side of the tear plane (See Figure 5 - (Right)).

- (c) Optionally, we may move the intersection points of the negative and positive sides in opposite directions so the actual tear becomes apparent and we re-calculate the weights of the new vertices (See Figure 5 - (Bottom)).

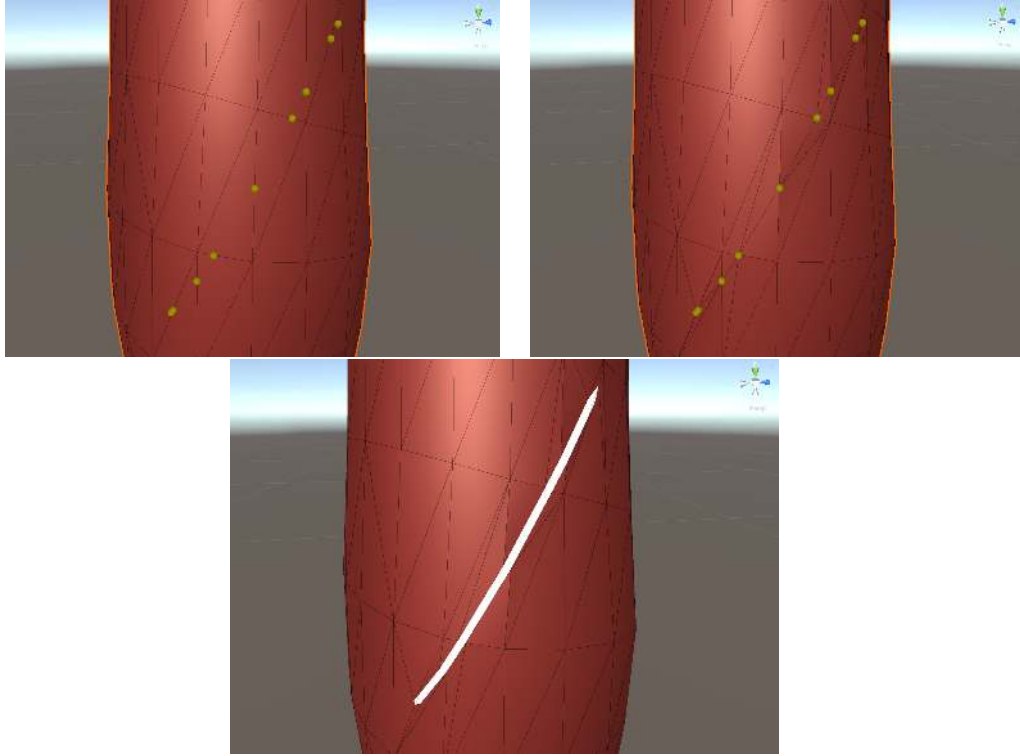


Figure 5: (Left) In between intersection points, (Right) Tear segment, (Bottom) Opened Tear segment.

3.1 Running Times for Tearing

All running times in this Section were produced on a PC with the following specs: CPU: Intel core i9 9900k @ 3.60GHz, GPU: Nvidia RTX 2080, RAM: 16GB.

3.1.1 Offline preprocessing running times

We are testing the following cases:

- **Case 1:** Regular non-dense model.

In this case we haven't applied any further triangulation to make the model more dense (i.e., we have the original arm model). Vertices: 2535, Triangles: 5037

- **Case 2:** Re-triangulated dense model.

In this case we have made the model more dense by splitting each of each triangles into four as we have described above. Vertices: 10109, Triangles: 20148.

Initialization running times (i.e., the time to calculate edge neighbours for each edge, make model more dense and optimize model by deleting duplicate vertices) (see Table 3):

	Make model dense	Optimize model	Calculate edge neighbors
Case1	-	-	1.184s
Case2	0.072s	0.2299s	74.8s

Table 3: Specific running times for each case for tearing.

3.1.2 Real-Time Tearing Running Times

We are testing the following cases (see Table 4):

- **Case 1:** Regular non-dense model.

In this case we haven't applied any further triangulation to make the model more dense(i.e. we have the original arm model). Vertices: 2535, Triangles: 5037.

- **Case 2:** Re-triangulated dense model.

In this case we have made the model more dense by splitting each of each triangles into four as we have described above (See Section 4.1). Vertices: 10109, Triangles: 20148.

	Case1	Case2
Initial tear points	9ms	29ms
All intersections	3ms	10.1ms
In-between intersections	2ms	4ms
Re-triangulate	0.18ms	0.28ms
Remake mesh	2.2ms	6.6ms
Total	19ms	50ms

Table 4: Specific running times for each case for tearing in real time.

4 Drilling Algorithm

4.1 Dense

The first step of the drilling algorithm is to make the model more dense. This procedure allows the circumference of the drilled area to have a sufficient number

of vertices and therefore the area drilled will look more like a circle rather than a polygon.

The dense function splits each triangle of the mesh that satisfies a certain condition into four smaller ones. The triangles chosen are the ones that have an edge smaller than a user-defined length (we usually set this length to be 30% of the average edge length). The re-triangulation is done as follows:

1. We find the middle point of each edge of the triangle (see Figure 6 - Left).
2. We connect the new vertices together to form a new triangle (see Figure 6 - Middle).
3. We delete the initial triangle and add another 3 as shown below (see Figure 6 - Right).

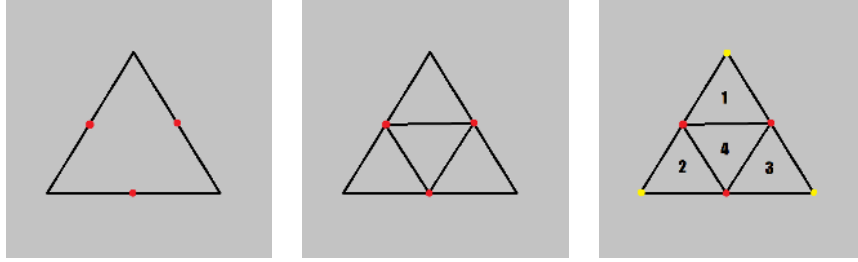


Figure 6: (Left) Middle point of each edge, (Middle) New triangle consisting of the new edges, (Right) New triangles.

4.2 Drill

After making the model more dense, the drilling process takes place. Initially, we are given a segment which is located in the center of our drilling circle and the radius of the desired circle. Then we perform the following steps:

1. First, we determine which vertices of our model are inside the drilling circle, which are outside and which are on the bounds of the circle. We store the indices of those vertices in three separate Lists. (see Figure 7 - (Left)).
2. Secondly, we find the triangles of each vertex that is inside the drilling circle and we filter out the ones that exist in the opposite side of the model since we want to drill only on the one side of it. We store the indices of such triangles in a list. (see Figure 7 - (Right)).
3. Afterwards, we iterate through all these triangles and find the intersection points on the edges of each triangle by projecting each point inside the

circle in a plane above and parallel to the circle plane. Then we find the intersection points between the edges in the plane that we have projected and the circle of the drilling. Then, we project those intersection points back to the model and we store them to a list. In this step, we also store the adjacent vertices of each intersection point to later calculate its weight. (see Figure 8 - (Left)).

4. Finally, after we have found the intersection points we retriangulate the drilling area, i.e., we remove the triangles inside the circle and we fix the ones that are close to the drilling area to include the new vertices(intersection points). Also we re-calculate the weights of the new vertices to allow further deformation (Kamarianakis & Papagiannakis, n.d.). (see Figure 8 - (Right)).

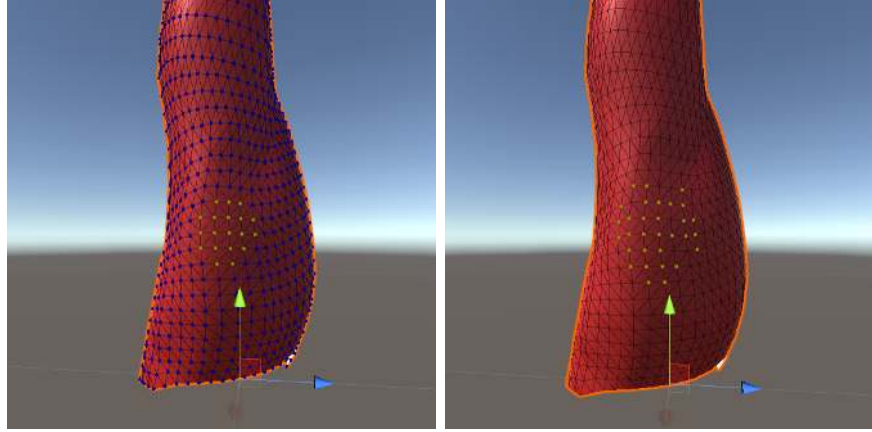


Figure 7: (Left) Vertex separation, (Right) Affected ids of the drilling.

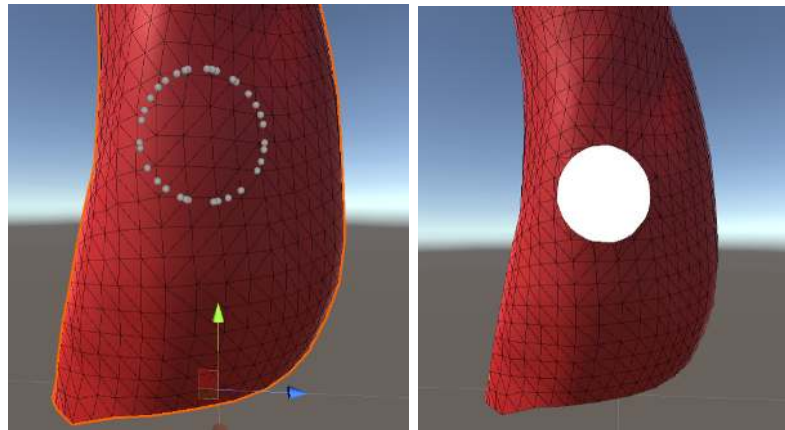


Figure 8: (Left) Drill intersection points, (Right) Complete drilled hole.

4.3 Running Times for Drilling

Specs: CPU: Intel core i7 7700HQ @ 2.80GHz, GPU: Nvidia GTX 1050 ti m, RAM: 8GB.

For the rest of the Section, "k" stands for the length that each edge of the triangle has to have in order to get re-triangulated and "times" stands for the times that the dense function will run.

We are testing the following cases (see Table 5):

- **Case 1:** Non-optimized no triangulation(k=1, times=0).

In this case we have made zero optimizations and we haven't applied any further triangulation to make the model more dense(i.e. we have the original arm model). Vertices: 2535, Triangles: 5037.

- **Case 2:** Non-optimized half triangulation(k=0.01, times=1).

In this case we have made zero optimizations but we have applied triangulation to make the model more dense to every edge of every triangle that has length greater than 0.01. Vertices: 3964, Triangles: 7692.

- **Case 3:** Non-optimized all triangulation(k=0, times=1).

In this case we have made zero optimizations but we have applied triangulation to make the model more dense to every edge of every triangle. Vertices: 10109, Triangles: 20148.

- **Case 4:** Non-optimized double all triangulation(k=0.002, times=2).

In this case we have made zero optimizations but we have applied triangulation to make the model more dense to every edge of every triangle, twice(i.e. the process of making the model more dense runs two times for every triangle in the model). Vertices: 26989, Triangles: 52482.

- **Case 5:** Optimized no triangulation(k=1, times=0).

In this case we have made optimizations, we optimized how we find the affected ids of the drilling on the arm by using a structure that holds the neighbors of each vertex so we don't have to iterate through the whole triangles array. By doing this we have moved some of the heavy work offline before the first frame is drawn(The timings for that are listed below). We haven't applied any further triangulation to make the model more dense(i.e. we have the original arm model). Vertices: 2535, Triangles: 5037.

- **Case 6:** Optimized half triangulation(k=0.01, times=1).

In this case we have made the same optimizations as Case 5(see above). We have applied triangulation to make the model more dense to every edge of every triangle that has length greater than 0.01. Vertices: 3964, Triangles: 7692.

- **Case 7:** Optimized all triangulation($k=0$, times=1).

In this case we have made the same optimizations as Case 5(see above). We have applied triangulation to make the model more dense to every edge of every triangle. Vertices: 10109, Triangles: 20148.

- **Case 8:** Optimized double all triangulation($k=0.002$, times=2).

In this case we have made the same optimizations as Case 5(see above). We have applied triangulation to make the model more dense to every edge of every triangle, twice. Vertices: 26989, Triangles: 52482.

Case1	Case2	Case3	Case4
2.2566s	27.69587s	30.29349s	404.1152s
Case5	Case6	Case7	Case8
0.4972992s	2.458544s	5.867214s	56.65717s

Table 5: The above running times are without the offline calculations that happen before the first frame(i.e. triangulation, neighbors, etc).

The offline time to calculate the neighbour of each vertex is: 6.459928s.

These are the running times for the specific function of the drilling (see Table 6):

- "Optimize" is the function that optimizes the model by deleting the duplicate vertices.
- "Ids Inside" is the function the detects and stores the vertices of the model that are inside the drilled are.
- "All Affected Ids" is the function that detects and stores all the ids of the triangles that each vertex inside of the drilled area belongs to.
- "Intersection" is the function that calculates and stores the intersections of the drilling circle with the model.
- "Triangulation" is the function that connects the new vertices/intersection points with the existing triangles of the model.

- "Split Faces" is the function that was described in detail in Section 4.1 and splits each triangle into four.

	Optimize	Ids inside	All affected ids
Case5	0.1863s	0.1011s	0.1800s
Case6	0.1866s	0.50750s	1.4442s
Case7	0.2186s	3.1663s	1.8115s
Case8	0.9928s	32.6252s	38.08816s
-	Intersection	Triangulation	Split Faces
Case5	0.1000s	0.00299s	0.01s
Case6	0.4586s	0.003365s	0.02s
Case7	0.51816s	0.004904s	0.07s
Case8	19.6365s	0.02821s	0.24389s

Table 6: Specific running times for each case.

- **Case 9:** We parallelized the detect Ids inside the drilled area, detect All affected ids of the drilling, and the Find All Neighbors in the init stage on the dense model (with $k = 0$ and times = 1), where every triangle gets more dense. (see Table 7): Vertices: 10109, Triangles: 20148.

The offline time to calculate the neighbour of each vertex in parallel is: 0.3134449s.

	Non-parallel-optimized	Parallel
Ids inside	2.8294s	0.001084s
All affected ids	1.493732s	0.001192s
Find All Neighbors	4.042238s	0.1260331s

Table 7: Specific running times on parallelized functions.

Here is the general execution time for the drilling (see Table 8):

	Non-parallel-optimized	Parallel
Drill	4.0892s	0.5034s

Table 8: Drilling running times.

5 Conclusions, Limitations and Future Work

In this work, we have presented algorithms that allow the user to perform: Cutting, Tearing and/or Drilling (CTD) on rigged 3D models that are fast enough(thanks

to parallelism and the use of custom data structures) to be used in real-time applications, while preserving the ability to re-deform and re-animate the model after the CTD.

The main limitations that we are currently facing regard the Cutting and Drilling Module. As shown in the showcase video, when the number of intersection points become larger than a threshold depending on the model, then we have small shutters due to increased running times required to perform the operation. Such problem usually arises when the user tries to perform a cut operation that affects a large portion of the model surface area or drills a whole with a quite large diameter.

In the future, we intend to optimize the performance of the Cut module by better utilizing parallelism. Using the edge neighbours list, similarly to the Tear module, we aim to further reduce the running times of the other two modules. Additionally, we plan to combine the CTD module with soft body 3D models that are also rigged in order to achieve improved and more realistic visual results.

References

- Bender, J., Müller, M., Otaduy, M. A., Teschner, M., & Macklin, M. (2014, September). A survey on position-based simulation methods in computer graphics. *Computer Graphics Forum*, 33(6), 228–251.
- Berndt, I. U., Torchelsen, R. P., & Maciel, A. (2017). Efficient Surgical Cutting with Position-Based Dynamics. *IEEE Computer Graphics and Applications*, 37(3), 24–31.
- Bielser, D., Glardon, P., Teschner, M., & Gross, M. (2004). A state machine for real-time cutting of tetrahedral meshes. In *11th pacific conference on computer graphics and applications* (pp. 377–386). IEEE Comput. Soc.
- Bielser, D., Maiwald, V. A., & Gross, M. H. (1999). Interactive Cuts through 3-Dimensional Soft Tissue. *Computer Graphics Forum*, 18(3), 31–38.
- Bruyns, C. D., & Senger, S. (2001, August). Interactive cutting of 3D surface meshes. *Computers & Graphics*, 25(4), 635–642.
- Bruyns, C. D., Senger, S., Menon, A., Montgomery, K., Wildermuth, S., & Boyle, R. (2002, February). A survey of interactive mesh-cutting techniques and a new method for implementing generalized interactive mesh cutting using virtual tools†. *The Journal of Visualization and Computer Animation*, 13(1), 21–42.
- Kamarianakis, M., & Papagiannakis, G. (n.d.). An all-in-one geometric algorithm for cutting, tearing, and drilling deformable models. *Advances in Applied Clifford Algebras*, 31(3), 58. Retrieved 2021-08-31, from <https://link.springer.com/10.1007/s00006-021-01151-6> doi: 10.1007/s00006-021-01151-6
- Magnenat-Thalmann, N., Laperrère, R., & Thalmann, D. (1988). Joint-dependent local deformations for hand animation and object grasping. In *In proceedings on graphics interface’88*.
- Mor, A. B., & Kanade, T. (2000). Modifying Soft Tissue Models: Progressive Cutting with Minimal New Element Creation. In *Advances in computer graphics* (pp. 598–607). Berlin, Heidelberg: Springer Berlin Heidelberg.

Wu, J., Westermann, R., & Dick, C. (2015, September). A Survey of Physically Based Simulation of Cuts in Deformable Bodies. *Computer Graphics Forum*, 34(6), 161–187.

A Manual and Instructions

Instructions on how to install the various programs that will be needed in order to run the code.

A.1 Unity Installation

How to install unity:

1. Go to their website and download and install the latest version on the unity hub (<https://unity3d.com/get-unity/download>) (see Figure 9).

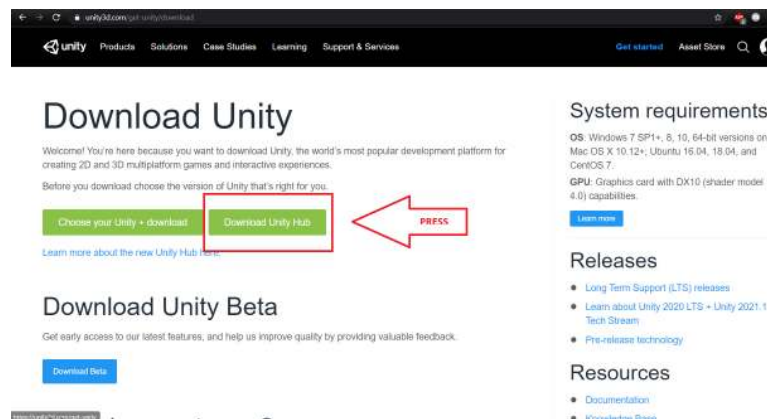


Figure 9: Press the button in the red rectangle and follow the instructions.

2. Open the unity hub and click on Installs to install a unity version. (see Figure 10).

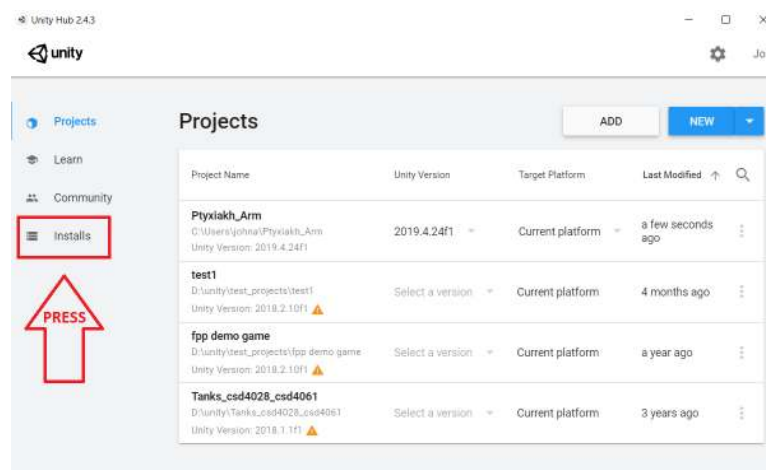


Figure 10: Press the button in the red rectangle to see the current installs.

3. Click on ADD to find which version to install. (see Figure 11).

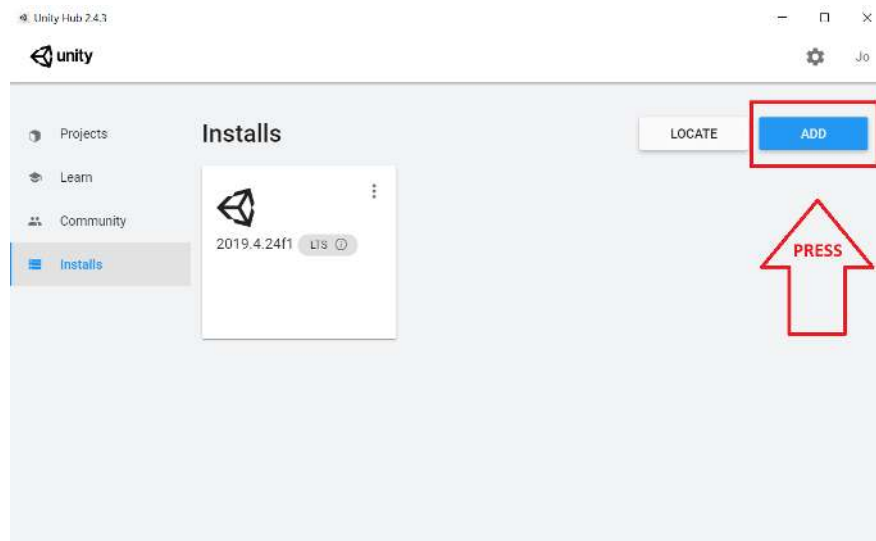


Figure 11: Press the button in the red rectangle to add a new Unity version.

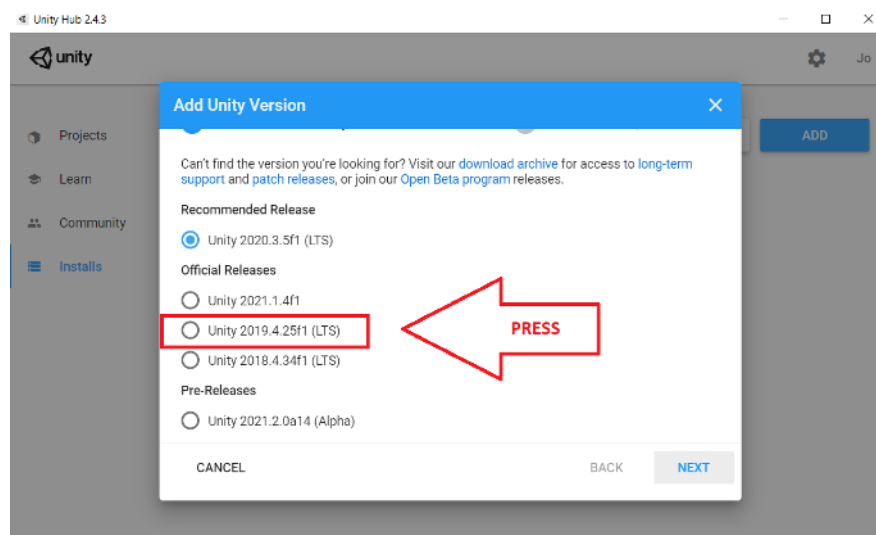


Figure 12: Press the button in the red rectangle to choose the correct version.

- Click on Unity 2019.4 (LTS) (the specific micro version may vary) and then click NEXT (see Figure 12).
- Click on DONE without choosing anything on the list and the installation should begin (see Figure 13).

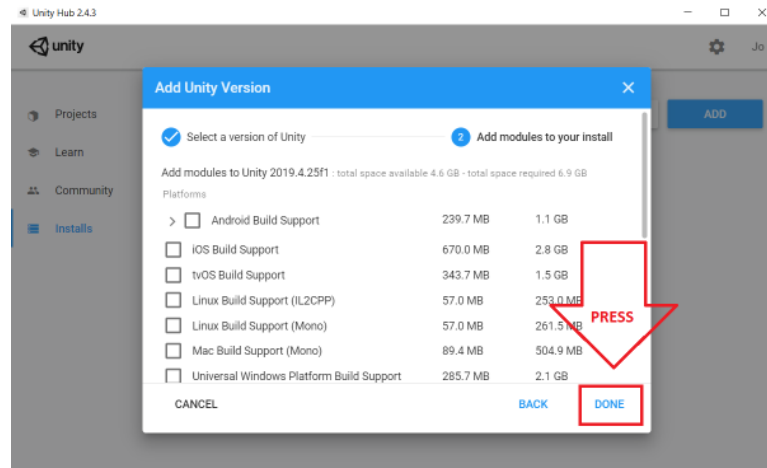


Figure 13: Press the button in the red rectangle to begin the installation.

A.2 Unity Project Creation

To create a project in unity:

- Click on Projects and then click on the arrow next to NEW (see Figure 14).

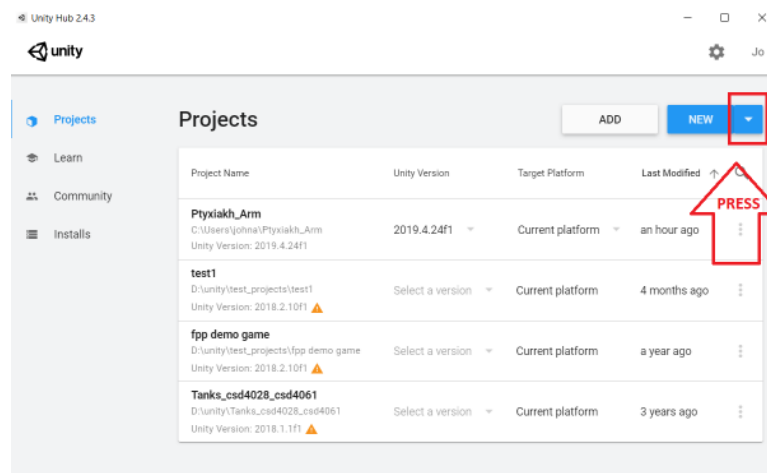


Figure 14: Press the button in the red rectangle to show available versions.

- Click on the unity version that we installed earlier (see Figure 15).
- Click on the 3D option on the left the fill in the name for the project and where to save it and lastly click on CREATE (see Figure 16).

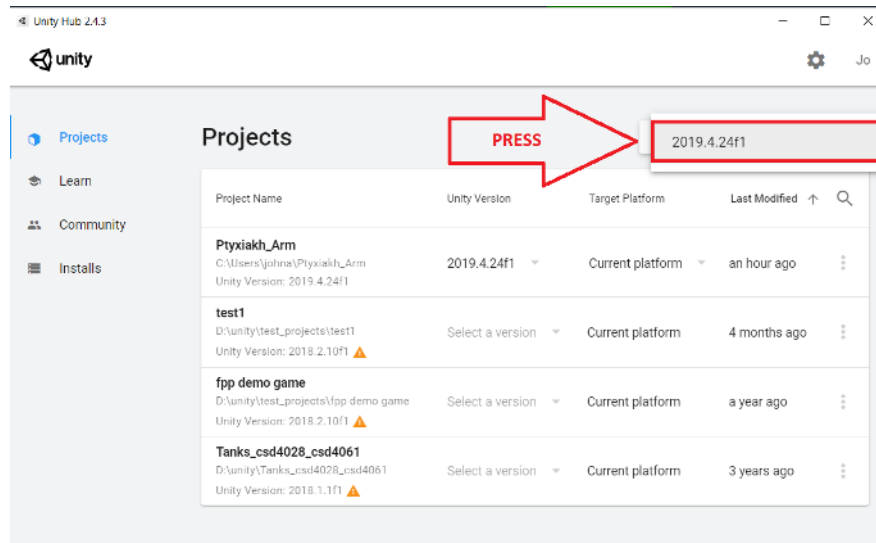


Figure 15: Press the button in the red rectangle to select the newly installed version.

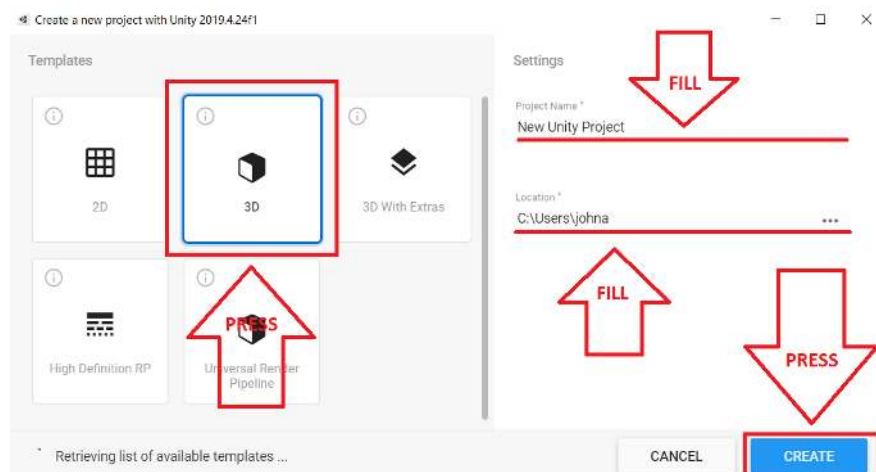


Figure 16: Create the project with the correct setting.

A.3 Import Models, Prefabs and Scripts Into Unity Project

To import the models, prefabs and scripts in unity:

1. Download and unzip the "CTD-Assets.zip" folder.
2. Open the folder and drag and drop the Scripts, Prefabs, and Models on the Project pane which is located on the bottom of the Unity editor (see Figure 17).

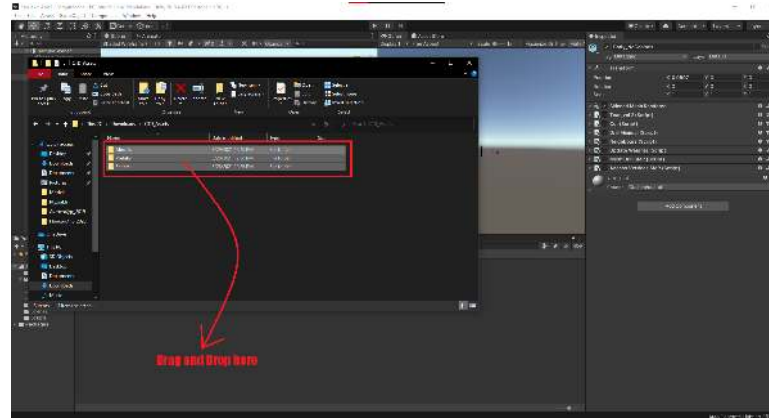


Figure 17: Drag and drop the assets inside unity.

3. Then, open the Prefabs folder inside Unity and drag and drop to the scene hierarchy the Arm5, DrillPivot, ScalpelPivot and KnifePivot (see Figure 18).

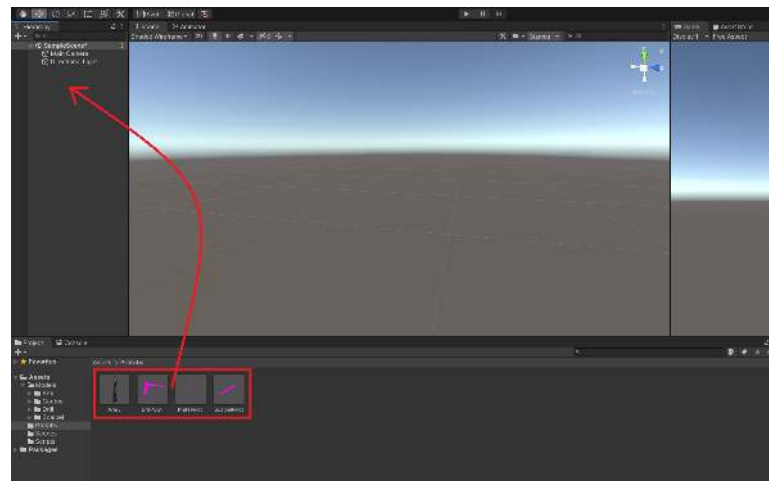


Figure 18: Drag and drop the prefabs into the hierarchy panel.

4. To perform the drilling, the DrillMinimal script on the Inspector has to be checked and the DrillPivot has to be in a position so that the tip of the drill is inside the arm model. Then, press play and press the key D to perform the drilling. By default, the drill is positioned in an appropriate position.

5. To perform the cutting, the Cut script on the Inspector has to be checked and the KnifePivot has to be in a position on the surface of the model that you want to cut. Then press play and hit the C key to perform the cutting. By default, the knife is positioned in an appropriate position.
6. To perform the tearing, the Tear vol 2 script on the Inspector has to be checked and the ScalpelPivot has to be in a position so that the tip of the blade is inside the arm model. Then, press play and hit the T key to perform the tearing. By default, the scalpel is positioned in an appropriate position.